JavaScript is disabled on your browser.

compact1, compact2, compact3

java.lang

Class ThreadGroup

java.lang.Object
    java.lang.ThreadGroup

All Implemented Interfaces:

Thread.UncaughtExceptionHandler

```
public class ThreadGroup
extends Object
implements Thread.UncaughtExceptionHandler
```

A thread group represents a set of threads. In addition, a thread group can also include other thread groups. The thread groups form a tree in which every thread group except the initial thread group has a parent.

A thread is allowed to access information about its own thread group, but not to access information about its thread group's parent thread group or any other thread groups.

Since:

JDK1.0

Constructor Summary

Constructors

Constructor	Description
`ThreadGroup(String name)`	Constructs a new thread group.
`ThreadGroup(ThreadGroup parent, String name)`	

	Creates a new thread group.

Method Summary

Deprecated Methods

Modifier and Type	Method	Description
int	**activeCount**()	Returns an estimate of the number of active threads in this thread group and its subgroups.
int	**activeGroupCount**()	Returns an estimate of the number of active groups in this thread grou and its subgroups.
boolean	**allowThreadSuspension**(boolean b)	**Deprecated.** The definition of this call depends on **suspend()**, which is deprecated. Further, the behavior of this call was never specified.
void	**checkAccess**()	Determines if the currently running thread has permission to modify thi thread group.
void	**destroy**()	Destroys this thread group and all of its subgroups.
int	**enumerate**(**Thread**[] list)	Copies into the specified array ever active thread in this thread group and its subgroups.
int	**enumerate**(**Thread**[] list, boolean recurse)	Copies into the specified array ever active thread in this thread group.
int	**enumerate**(**ThreadGroup**[] list)	Copies into the specified array references to every active subgrou in this thread group and its subgroups.
int	**enumerate**(**ThreadGroup**[] list, boolean recurse)	Copies into the specified array references to every active subgrou in this thread group.
int	**getMaxPriority**()	Returns the maximum priority of this thread group.

String	getName()	Returns the name of this thread group.
ThreadGroup	getParent()	Returns the parent of this thread group.
void	interrupt()	Interrupts all threads in this thread group.
boolean	isDaemon()	Tests if this thread group is a daemon thread group.
boolean	isDestroyed()	Tests if this thread group has been destroyed.
void	list()	Prints information about this thread group to the standard output.
boolean	parentOf(ThreadGroup g)	Tests if this thread group is either the thread group argument or one of its ancestor thread groups.
void	resume()	**Deprecated.** This method is used solely in conjunction with `Thread.suspend` and `ThreadGroup.suspend`, both of which have been deprecated, as they are inherently deadlock-prone. See `Thread.suspend()` for details.
void	setDaemon(boolean daemon)	Changes the daemon status of this thread group.
void	setMaxPriority(int pri)	Sets the maximum priority of the group.
void	stop()	**Deprecated.** This method is inherently unsafe. See `Thread.stop()` for details.
void	suspend()	**Deprecated.** This method is inherently deadlock-prone. See `Thread.suspend()` for details.
String	toString()	Returns a string representation of this Thread group.

void	**uncaughtException**(**Thread** t, **Throwable** e)	Called by the Java Virtual Machine when a thread in this thread group stops because of an uncaught exception, and the thread does not have a specific **Thread.UncaughtExceptionHandle** installed.

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

Constructor Detail

ThreadGroup

```
public ThreadGroup(String name)
```

Constructs a new thread group. The parent of this new group is the thread group of the currently running thread.

The checkAccess method of the parent thread group is called with no arguments; this may result in a security exception.

Parameters:

name - the name of the new thread group.

Throws:

SecurityException - if the current thread cannot create a thread in the specified thread group.

Since:

JDK1.0

See Also:

checkAccess()

ThreadGroup

```
public ThreadGroup(ThreadGroup parent,
                   String name)
```

Creates a new thread group. The parent of this new group is the specified thread group.

The checkAccess method of the parent thread group is called with no arguments; this may result in a security exception.

Parameters:

parent - the parent thread group.

name - the name of the new thread group.

Throws:

NullPointerException - if the thread group argument is null.

SecurityException - if the current thread cannot create a thread in the specified thread group.

Since:

JDK1.0

See Also:

SecurityException, checkAccess()

Method Detail

getName

public final String getName()

Returns the name of this thread group.

Returns:

the name of this thread group.

Since:

JDK1.0

getParent

public final ThreadGroup getParent()

Returns the parent of this thread group.

First, if the parent is not null, the checkAccess method of the parent thread group is called with no arguments; this may result in a security exception.

Returns:

the parent of this thread group. The top-level thread group is the only thread group whose parent is null.

Throws:

SecurityException - if the current thread cannot modify this thread group.

Since:

JDK1.0

See Also:

checkAccess(), SecurityException, RuntimePermission

getMaxPriority

```
public final int getMaxPriority()
```

Returns the maximum priority of this thread group. Threads that are part of this group cannot have a higher priority than the maximum priority.

Returns:

the maximum priority that a thread in this thread group can have.

Since:

JDK1.0

See Also:

setMaxPriority(int)

isDaemon

```
public final boolean isDaemon()
```

Tests if this thread group is a daemon thread group. A daemon thread group is automatically destroyed when its last thread is stopped or its last thread group is destroyed.

Returns:

true if this thread group is a daemon thread group; false otherwise.

Since:

JDK1.0

isDestroyed

```
public boolean isDestroyed()
```

Tests if this thread group has been destroyed.

Returns:

true if this object is destroyed

Since:

JDK1.1

setDaemon

```
public final void setDaemon(boolean daemon)
```

Changes the daemon status of this thread group.

First, the `checkAccess` method of this thread group is called with no arguments; this may result in a security exception.

A daemon thread group is automatically destroyed when its last thread is stopped or its last thread group is destroyed.

Parameters:

`daemon - if true, marks this thread group as a daemon thread group; otherwise, marks this thread group as normal.`

Throws:

`SecurityException - if the current thread cannot modify this thread group.`

Since:

JDK1.0

See Also:

`SecurityException`, `checkAccess()`

setMaxPriority

```
public final void setMaxPriority(int pri)
```

Sets the maximum priority of the group. Threads in the thread group that already have a higher priority are not affected.

First, the `checkAccess` method of this thread group is called with no arguments; this may result in a security exception.

If the `pri` argument is less than `Thread.MIN_PRIORITY` or greater than `Thread.MAX_PRIORITY`, the maximum priority of the group remains unchanged.

Otherwise, the priority of this ThreadGroup object is set to the smaller of the specified `pri` and the maximum permitted priority of the parent of this thread group. (If this thread group is the system thread group, which has no parent, then its maximum priority is simply set to `pri`.) Then this method is called recursively, with `pri` as its argument, for every thread group that belongs to this thread group.

Parameters:

pri - the new priority of the thread group.

Throws:

SecurityException - if the current thread cannot modify this thread group.

Since:

JDK1.0

See Also:

getMaxPriority(), SecurityException, checkAccess()

parentOf

```
public final boolean parentOf(ThreadGroup g)
```

Tests if this thread group is either the thread group argument or one of its ancestor thread groups.

Parameters:

g - a thread group.

Returns:

true if this thread group is the thread group argument or one of its ancestor thread groups; false otherwise.

Since:

JDK1.0

checkAccess

```
public final void checkAccess()
```

Determines if the currently running thread has permission to modify this thread group.

If there is a security manager, its `checkAccess` method is called with this thread group as its argument. This may result in throwing a `SecurityException`.

Throws:

`SecurityException - if the current thread is not allowed to access this thread group.`

Since:

`JDK1.0`

See Also:

`SecurityManager.checkAccess(java.lang.ThreadGroup)`

activeCount

```
public int activeCount()
```

Returns an estimate of the number of active threads in this thread group and its subgroups. Recursively iterates over all subgroups in this thread group.

The value returned is only an estimate because the number of threads may change dynamically while this method traverses internal data structures, and might be affected by the presence of certain system threads. This method is intended primarily for debugging and monitoring purposes.

Returns:

`an estimate of the number of active threads in this thread group and in any other thread group that has this thread group as an ancestor`

Since:

`JDK1.0`

enumerate

```
public int enumerate(Thread[] list)
```

Copies into the specified array every active thread in this thread group and its subgroups.

An invocation of this method behaves in exactly the same way as the invocation

```
enumerate(list, true)
```

Parameters:

`list - an array into which to put the list of threads`

Returns:

`the number of threads put into the array`

Throws:

`SecurityException - if checkAccess determines that the current thread cannot access this thread group`

Since:

`JDK1.0`

enumerate

```
public int enumerate(Thread[] list,
                     boolean recurse)
```

Copies into the specified array every active thread in this thread group. If `recurse` is `true`, this method recursively enumerates all subgroups of this thread group and references to every active thread in these subgroups are also included. If the array is too short to hold all the threads, the extra threads are silently ignored.

An application might use the activeCount method to get an estimate of how big the array should be, however *if the array is too short to hold all the threads, the extra threads are silently ignored.* If it is critical to obtain every active thread in this thread group, the caller should verify that the returned int value is strictly less than the length of `list`.

Due to the inherent race condition in this method, it is recommended that the method only be used for debugging and monitoring purposes.

Parameters:

`list - an array into which to put the list of threads`

`recurse - if true, recursively enumerate all subgroups of this thread group`

Returns:

`the number of threads put into the array`

Throws:

`SecurityException - if checkAccess determines that the current thread cannot access this thread group`

Since:

JDK1.0

activeGroupCount

```
public int activeGroupCount()
```

Returns an estimate of the number of active groups in this thread group and its subgroups. Recursively iterates over all subgroups in this thread group.

The value returned is only an estimate because the number of thread groups may change dynamically while this method traverses internal data structures. This method is intended primarily for debugging and monitoring purposes.

Returns:

the number of active thread groups with this thread group as an ancestor

Since:

JDK1.0

enumerate

```
public int enumerate(ThreadGroup[] list)
```

Copies into the specified array references to every active subgroup in this thread group and its subgroups.

An invocation of this method behaves in exactly the same way as the invocation

```
enumerate(list, true)
```

Parameters:

list - an array into which to put the list of thread groups

Returns:

the number of thread groups put into the array

Throws:

SecurityException - if checkAccess determines that the current thread cannot access this thread group

Since:

JDK1.0

enumerate

```
public int enumerate(ThreadGroup[] list,
                     boolean recurse)
```

Copies into the specified array references to every active subgroup in this thread group. If `recurse` is `true`, this method recursively enumerates all subgroups of this thread group and references to every active thread group in these subgroups are also included.

An application might use the activeGroupCount method to get an estimate of how big the array should be, however *if the array is too short to hold all the thread groups, the extra thread groups are silently ignored.* If it is critical to obtain every active subgroup in this thread group, the caller should verify that the returned int value is strictly less than the length of `list`.

Due to the inherent race condition in this method, it is recommended that the method only be used for debugging and monitoring purposes.

Parameters:

`list - an array into which to put the list of thread groups`

`recurse - if true, recursively enumerate all subgroups`

Returns:

`the number of thread groups put into the array`

Throws:

`SecurityException - if checkAccess determines that the current thread cannot access this thread group`

Since:

`JDK1.0`

stop

```
@Deprecated
public final void stop()
```

Deprecated. *This method is inherently unsafe. See `Thread.stop()` for details.*

Stops all threads in this thread group.

First, the `checkAccess` method of this thread group is called with no arguments; this may result in a security exception.

This method then calls the `stop` method on all the threads in this thread group and in all of its subgroups.

Throws:

SecurityException - if the current thread is not allowed to access this thread group or any of the threads in the thread group.

Since:

JDK1.0

See Also:

SecurityException, Thread.stop(), checkAccess()

interrupt

```
public final void interrupt()
```

Interrupts all threads in this thread group.

First, the checkAccess method of this thread group is called with no arguments; this may result in a security exception.

This method then calls the interrupt method on all the threads in this thread group and in all of its subgroups.

Throws:

SecurityException - if the current thread is not allowed to access this thread group or any of the threads in the thread group.

Since:

1.2

See Also:

Thread.interrupt(), SecurityException, checkAccess()

suspend

```
@Deprecated
public final void suspend()
```

Deprecated. *This method is inherently deadlock-prone. See Thread.suspend() for details.*

Suspends all threads in this thread group.

First, the checkAccess method of this thread group is called with no arguments; this may result in a security exception.

This method then calls the suspend method on all the threads in this thread group and in all of its subgroups.

Throws:

SecurityException - if the current thread is not allowed to access this thread group or any of the threads in the thread group.

Since:

JDK1.0

See Also:

`Thread.suspend()`, `SecurityException`, `checkAccess()`

resume

```
@Deprecated
public final void resume()
```

Deprecated. *This method is used solely in conjunction with `Thread.suspend` and `ThreadGroup.suspend`, both of which have been deprecated, as they are inherently deadlock-prone. See `Thread.suspend()` for details.*

Resumes all threads in this thread group.

First, the `checkAccess` method of this thread group is called with no arguments; this may result in a security exception.

This method then calls the `resume` method on all the threads in this thread group and in all of its sub groups.

Throws:

`SecurityException` - if the current thread is not allowed to access this thread group or any of the threads in the thread group.

Since:

JDK1.0

See Also:

`SecurityException`, `Thread.resume()`, `checkAccess()`

destroy

```
public final void destroy()
```

Destroys this thread group and all of its subgroups. This thread group must be empty, indicating that all threads that had been in this thread group have since stopped.

First, the `checkAccess` method of this thread group is called with no arguments; this may result in a security exception.

Throws:

`IllegalThreadStateException` - if the thread group is not empty or if the thread group has already been destroyed.

`SecurityException` - if the current thread cannot modify this thread group.

Since:

JDK1.0

See Also:

checkAccess()

list

```
public void list()
```

Prints information about this thread group to the standard output. This method is useful only for debugging.

Since:

JDK1.0

uncaughtException

```
public void uncaughtException(Thread t,
                              Throwable e)
```

Called by the Java Virtual Machine when a thread in this thread group stops because of an uncaught exception, and the thread does not have a specific Thread.UncaughtExceptionHandler installed.

The uncaughtException method of ThreadGroup does the following:

- If this thread group has a parent thread group, the uncaughtException method of that parent is called with the same two arguments.
- Otherwise, this method checks to see if there is a default uncaught exception handler installed, and if so, its uncaughtException method is called with the same two arguments.
- Otherwise, this method determines if the Throwable argument is an instance of ThreadDeath. If so, nothing special is done. Otherwise, a message containing the thread's name, as returned from the thread's getName method, and a stack backtrace, using the Throwable's printStackTrace method, is printed to the standard error stream.

Applications can override this method in subclasses of ThreadGroup to provide alternative handling of uncaught exceptions.

Specified by:

uncaughtException in interface Thread.UncaughtExceptionHandler

Parameters:

t - the thread that is about to exit.

e - the uncaught exception.

Since:

JDK1.0

allowThreadSuspension

```
@Deprecated
public boolean allowThreadSuspension(boolean b)
```

Deprecated. *The definition of this call depends on `suspend()`, which is deprecated. Further, the behavior of this call was never specified.*

Used by VM to control lowmem implicit suspension.

Parameters:

`b - boolean to allow or disallow suspension`

Returns:

`true on success`

Since:

JDK1.1

toString

```
public String toString()
```

Returns a string representation of this Thread group.

Overrides:

`toString` in class `Object`

Returns:

`a string representation of this thread group.`

Since:

JDK1.0

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.